D6 Verifying AI output

A workflow to check AI code before you trust it.

D6-L1 Read before you run, and check each API

Restate the goal, list and check every name, hunt for side effects, then run on development data.

Five steps of the read-through	Restate the goal, list names, check each name, find side effects, run on dev data.
Side effects to hunt for	mode('overwrite'), DROP, DELETE, TRUNCATE, VACUUM, writes to production names.
Quick existence checks in a notebook	hasattr for names, inspect.signature(...).parameters for parameters, DESCRIBE TABLE or df.columns for columns.
What a read-through does not do	Prove correctness. It clears visible mistakes so tests can find the rest.

D6-L2 Small data with a known answer, failing test first

Hand-built data, hand-worked answers, a test that fails first, and a deliberate break to prove the test bites.

Known-answer data	5 to 10 hand-built rows: normal case, duplicate, null, zero total, empty group, boundary.
Test-first order	Write the test, watch it fail, accept the code, watch it pass, break the code on purpose, watch it fail.
assertDataFrameEqual defaults (PySpark 4.0)	checkRowOrder=False, rtol=1e-05, atol=1e-08.
Why break code on purpose	If the test still passes, it misses that rule. A quick form of mutation testing.

D6-L3 Ask for assumptions, then compare a second approach

Make the assistant state its assumptions, check them on real data, keep the important ones as checks, and confirm results by an independent route.

Prompt for assumptions	'List every assumption this code makes about its input data, one per line.' Then check each on real data.
Common data assumptions to check	Unique keys per grain, non-null positive weights, id formats, UTC timestamps, one lookup row per key.
An independent second approach	SQL vs DataFrame code, pandas vs Spark on an extract, gold totals vs silver aggregates. Not the same assistant rewriting itself.
When two methods disagree	Run both on known-answer data. The one matching the hand-worked answer is right.